Lab 9

Objective: Use pointers to read data from stdin

This lab builds on lectures 8 and 9

Exercise 1: Hello World

The first program most programmers write in a new language is "hello world". This program is largely to test the compiler. Several IDEs are available on the lab computers: Visual Studio Code, Code Blocks, JGrasp, Visual Studio. Alternatively, you could opt for simpler text editors such as vim or Notepad++, and compile/run your code using a terminal.

Here's the program (save it in hello_world.c), you don't need to enter the comments:

```
#include <stdio.h>                  // needed to call printf()
int main(void)
      {
      printf("Hello World.\n");
      return 0;                     // return 0 (success) exit to the OS
      }
```

To compile it, run:

```
gcc -Wall -Wextra hello_world.c -o hello_world
```

That means "compile, enable extra warnings, my source code is in `hello_world.c` and the program to run should be called `hello_world`.

To execute it, run:

```
./hello_world
```

Exercise 2: Looping with a for loop

Write a program in C that outputs the integers from 1 to 7 along with the square of that integer. This will require you to declare some variables, use a `for` loop, and use `printf()` to print information to the terminal window.

You can use `printf()` to print 2 integers, `a`, and `b` like this:

```
printf("%d %d\n", a, b);
```

The output of your program should look something like this:

```
1 1
2 4
3 9
4 16
5 25
6 36
7 49
```

Exercise 3: Mean of a set of numbers
There are several ways to read input in C, one easy way is to use `scanf()`, so that's what we'll use. The C run time library is written in C. That means `scanf()` is written in C – something to think about when using it.

When we wanted to print an integer we wrote something like:

```
printf("%d", x);
```

To read an integer we do nearly the same, but with scanf().

```
scanf("%d", &x);
```

Note the use of `&`, to take the address of a variable. When we pass `&x` to `scanf()` what we're saying is "put the answer into `x`". Just like a reference in Java, but in C we are explicit about it being a pointer.

To read a `double` we use the format specifier `%lf`, and pass the address of a `double` to `scanf()`.

Write a program that reads 10 integers from stdin (the terminal) and prints the average (the mean) on the screen.

To read more than one value from the same line, use multiple format strings. So

```
scanf("%d %lf", &x, &y);
```

Will read an integer and a double.

Exercise 4: Average of middle scores
In some sporting competitions (such as figure skating) multiple judges score the competitor. But to avoid bias the top and bottom scores are thrown away – the final score is the average of the remaining scores. We're going to write the program that does this.

The input is one competitor per line, 5 scores per line (one for each judge). The task is to write the program that works out which competitor wins and what their score was. Hint: `scanf()` returns the number of input items successfully returned, so if you read 5 doubles in a single line and `scanf()` doesn't return 5 then something went wrong (in this case, you're at the end of the input). You might choose to use an array for this exercise.

For the input:

```
5.5 5.7 5.9 5.3 6.0
5.4 5.4 5.4 5.4 5.4
6.1 6.5 6.3 7.1 7.2
4.4 4.6 6.0 1.0 9.0
```

The output should be:

```
Winner is competitor 3 with a score of 6.633333
```

Exercise 5: We're got this sorted!
The task for this exercise is to explain the result in the previous exercise. We're going to read the input for a single competitor, sort it, compute their score, and write out the competitor number, their score, then a sorted list of the judges scores. For the data in the previous exercise, we expect to see something like this:

```
Competitor 1 scores 5.700000 from 5.300000 5.500000 5.700000 5.900000 6.000000
Competitor 2 scores 5.400000 from 5.400000 5.400000 5.400000 5.400000 5.400000
Competitor 3 scores 6.633333 from 6.100000 6.300000 6.500000 7.100000 7.200000
Competitor 4 scores 5.000000 from 1.000000 4.400000 4.600000 6.000000 9.000000
Winner is competitor 3 with a score of 6.633333
```

To make this easier write a `swap()` routine that swaps 2 `doubles`. Then write a `sort()` routine that sorts 5 doubles. `sort()` isn't as hard as it sounds. I'd do this by putting the values into an array and then implementing a bubble sort (which is efficient when n is small).